Chain of Responsibility

Chain · Route support tickets billing→tech→general · Build with Waqar

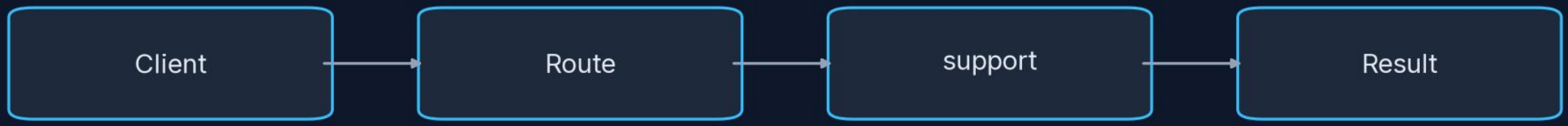
Today's Hook

- Problem: Route support tickets billing→tech→general
- Pattern: Chain — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Route support tickets billing→tech→general
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Chain: Route support tickets billing→tech→general

Real-World Scenario

- Running example: Route support tickets billing→tech→general
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.chain.problem;

import java.util.*;

public final class NaiveRouter {
  public static String route(String type){
    if("billing".equals(type)) return "billing-desk";
    if("tech".equals(type)) return "tech-desk";
    return "general-desk";
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.gof.behavioral.chain.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;

abstract class Handler {
  private Handler next;
  Handler link(Handler n){ next=n; return n; }
  final String handle(String type){
    String r = doHandle(type);
    if (r!=null) return r;
    return next==null? "unhandled": next.handle(type);
  }
  abstract String doHandle(String type);
}
final class BillingHandler extends Handler { String doHandle(String t){ return "billing".equals(t)? "billing-desk": null; } }
final class TechHandler extends Handler { String doHandle(String t){ return "tech".equals(t)? "tech-desk": null; } }
final class GeneralHandler extends Handler { String doHandle(String t){ return "general-desk"; } }
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.behavioral.chain.problem;

import java.util.*;
import java.util.concurrent.*;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Chain: support ticket routing ===");
    System.out.println("--- naive ---");
System.out.println(NaiveRouter.route("billing"));
System.out.println("--- chain ---");
var h=new BillingHandler(); h.link(new TechHandler()).link(new GeneralHandler());
System.out.println(h.handle("billing"));
System.out.println(h.handle("tech"));
System.out.println(h.handle("other"));
  }
}
```

Walkthrough Output

```
=== Chain: support ticket routing ===
--- naive ---
billing-desk
--- chain ---
billing-desk
tech-desk
general-desk
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Chain.

Related Patterns

- Contrast Chain with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Chain → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.